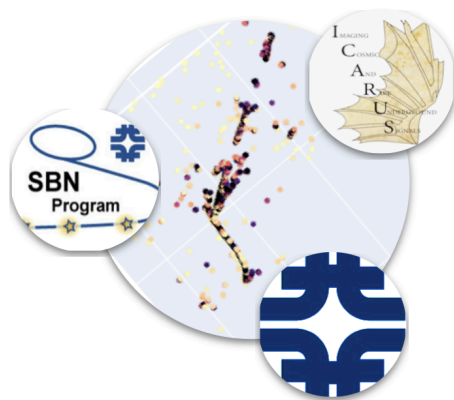


# NUGRAPH2 PORT GUIDE

v10\_06\_00\_01p01 → v10\_06\_00\_04p04

ICARUS - NUGRAPH2 GNN EXTENSIONS TO 2D DECONVOLUTION DATA/MCS

S. DEY | APRIL 2026



Thank you to Giuseppe Cerati for help with debugging!.

## ABOUT:

This document describes the complete port of the NuGraph2 GNN extensions from icaruscode **v10\_06\_00\_01p01** (1D deconvolution) to **v10\_06\_00\_04p04** (2D deconvolution / WireCell). It covers all seven repositories, the specific changes required in each, common errors encountered, and their fixes.

REPOSITORIES: larrecodnn, lardataobj, larpandoracontent, larpandora, sbnanaobj, icaruscode, sbncode.

—

PART A | Higher-level overview, mapping out what to do & in what order, details of python notebooks

PART B | Full details of port

PART C | Post-port fixes (most difficult, 1D → 2D deconvolution adjustments to base code)

-

APPENDIX A | NuGraph as a part of the ICARUS/ SBN reconstruction framework

APPENDIX B | grep cheatsheet

APPENDIX C | gdb cheatsheet



## A1. HIGHER-LEVEL OVERVIEW AND CONTEXT

### 1.1 WHAT IS BEING PORTED

The ICARUS experiment uses a Pandora-based reconstruction pipeline that runs in stages:

| Stage   | Description                                          | Output         |
|---------|------------------------------------------------------|----------------|
| Stage 0 | Raw detector data                                    | Raw ROOT file  |
| Stage 1 | Reconstruction: clustering, Pandora, tracks, showers | Reco ROOT file |
| CAF     | Common Analysis Format filling from Stage 1 products | Flat ROOT file |

NuGraph2 is a Graph Neural Network that runs inside the Stage 1 pipeline. It assigns semantic labels (MIP, HIP, Shower, Michel, Diffuse) and a filter score to individual TPC hits, then feeds a second-pass Pandora reconstruction using only neutrino-tagged hits.

The NuGraph2 extensions were developed against [v10\\_06\\_00\\_01p01](#) (1D deconvolution). The goal of this port is to make the same NuGraph2 functionality available in [v10\\_06\\_00\\_04p04](#) (2D deconvolution / WireCell).

### 1.2 THE TWO-LAYER PROBLEM

Before starting any porting work, it is essential to understand that there are two independent layers of change stacked on each other:

| Layer          | What It Is                            | Key Changes                                                      |
|----------------|---------------------------------------|------------------------------------------------------------------|
| <b>Layer 1</b> | Base version upgrade: v01p01 → v04p04 | 2D deconvolution, WireCell, combineHits producers, new CAF trees |
| <b>Layer 2</b> | NuGraph2 patch on top of Layer 1      | New modules, FCL sequences, CAF variables, filling logic         |

The NuGraph patch was written assuming Layer 1 does not exist. Wherever the NuGraph code references stage1 product labels, data structures, or module names that changed in the base version upgrade, it will fail even if the patch applies cleanly. The diagnostic methodology in Section 3 characterises Layer 1 before attempting Layer 2.

## A2. REPOSITORIES INVOLVED

---

Seven repositories carry the NuGraph2 changes. They are listed in dependency order (most upstream first). This order determines the correct port sequence.

| # | Repository        | Role in NuGraph Pipeline                                             | Fork / Location            | Port Order      |
|---|-------------------|----------------------------------------------------------------------|----------------------------|-----------------|
| 1 | larpandoracontent | Pure Pandora C++ - adds m_hitScores / m_hitScoreLabels to LArCaloHit | rtriozzi/larpandoracontent | First           |
| 2 | lardataobj        | LArSoft data object definitions - NuGraph class additions            | leonardo-lena/lardataobj   | Skip (upstream) |
| 3 | larrecodnn        | Base NuGraph2 inference framework                                    | leonardo-lena/larrecodnn   | Skip (upstream) |
| 4 | larpandora        | LArSoft↔Pandora interface - collects NuGraph hit labels              | rtriozzi/larpandora        | Second          |
| 5 | icaruscode        | ICARUS-specific modules, FCL sequences, stage1 integration           | SBNSoftware/icaruscode     | Third           |
| 6 | sbnanaobj         | CAF data object definitions - SRNuGraphSliceInfo, SRSlice additions  | SBNSoftware/sbnanaobj      | Fourth          |
| 7 | sbncode           | CAF filling logic - FillSliceNuGraph, FillPFPNuGraph                 | SBNSoftware/sbncode        | Fifth           |

### A3. DIAGNOSTIC METHODOLOGY

Run all four diagnostic steps before touching any code. These steps tell you exactly what work is needed and prevent surprises during the port.

#### 3.1 CHECK DEPENDENCY VERSIONS UNDER BOTH BASE RELEASES

Run in a terminal (not Jupyter - env vars do not persist across Jupyter cells):

```
# v01p01 dependency versions
source /cvmfs/icarus.opensciencegrid.org/products/icarus/setup_icarus.sh
setup icaruscode v10_06_00_01p01 -q e26:prof
echo "SBNANAOBJ=$SBNANAOBJ_VERSION"
echo "SBNCODE=$SBNCODE_VERSION"
echo "LARRECODNN=$LARRECODNN_VERSION"
echo "LARPANDORA=$LARPANDORA_VERSION"
echo "LARPANDORACONTENT=$LARPANDORACONTENT_VERSION"
```

```
# v04p04 dependency versions (new terminal)
source /cvmfs/icarus.opensciencegrid.org/products/icarus/setup_icarus.sh
setup icaruscode v10_06_00_04p04 -q e26:prof
echo "SBNANAOBJ=$SBNANAOBJ_VERSION"
echo "SBNCODE=$SBNCODE_VERSION"
echo "LARRECODNN=$LARRECODNN_VERSION"
echo "LARPANDORA=$LARPANDORA_VERSION"
echo "LARPANDORACONTENT=$LARPANDORACONTENT_VERSION"
```

Reference output for this port:

| Package           | v01p01 Version | v04p04 Version | Changed?  | Implication                                  |
|-------------------|----------------|----------------|-----------|----------------------------------------------|
| sbnanaobj         | v10_00_04      | v10_00_05      | Yes       | 1 patch bump - minor context drift possible  |
| sbncode           | v10_06_00_01   | v10_06_00_04   | Yes       | 3 patch bumps - manual porting may be needed |
| larrecodnn        | v10_01_10      | v10_01_10      | Identical | Zero context drift - patch applies directly  |
| larpandora        | v10_00_19      | v10_00_19      | Identical | Zero context drift - patch applies directly  |
| larpandoracontent | v04_15_01      | v04_15_01      | Identical | Zero context drift - patch applies directly  |

*Note: Identical versions mean the patch applies against byte-for-byte identical base code.*

#### 3.2 GENERATE DIFFS FOR ALL 7 REPOS

Use the notebook [all\\_package\\_diffs\\_cells.ipynb](#) with version strings hardcoded (not from env vars). The notebook generates `full_diff_<repo>.patch` for each package.

Summary of diff scope (reference output for this port):

| Repo              | Files Changed | Patch Size | Risk        | Key Notes                                                               |
|-------------------|---------------|------------|-------------|-------------------------------------------------------------------------|
| icaruscode        | 38            | 163.6 KB   | Medium-High | Bulk is net-new NuGraph dir; existing FCL edits are the risk            |
| sbncode           | 4             | 22.7 KB    | Medium-High | CAF filling logic - API signature refactor                              |
| larpandora        | 6             | 11.0 KB    | Medium      | Surgical C++ changes to LArPandoraInterface/                            |
| larpandoracontent | 1             | 9.6 KB     | Low         | Single header LArCaloHit.h - purely additive                            |
| sbnnanobj         | 5             | 5.6 KB     | Low-Medium  | New SRNuGraphSliceInfo class + SRSlice additions; watch classes_def.xml |
| lardataobj        | 0             | 0.0 KB     | None        | Already merged upstream - verify with git log                           |
| larrecodnn        | 0             | 0.0 KB     | None        | Already merged upstream - verify with git log                           |

### 3.3 CAF VARIABLE COMPARISON (CAFVARDIFFBETWEENVERSIONS.IPYNB)

Run the notebook pointing at one vanilla v01p01 CAF file and one vanilla v04p04 CAF file. The key output is the pairwise branch presence table.

Reference output:

| Comparison       | Branches in A | Branches in B | Only in A | Only in B | Jaccard Similarity |
|------------------|---------------|---------------|-----------|-----------|--------------------|
| v01p01 vs v04p04 | 2057          | 2106          | 0         | 49        | 0.977              |
| v01p01 vs v06p03 | 2057          | 2194          | 0         | 137       | 0.938              |
| v04p04 vs v06p03 | 2106          | 2194          | 0         | 88        | 0.960              |

Critical finding:  $n_{\text{only\_A}} = 0$  in every comparison.

No CAF branch was removed or renamed between any of these versions. Every v01p01 variable exists identically in v04p04. The 49 new branches in v04p04 are pure additions. This means the NuGraph CAF filling code in sbncode requires no variable name changes whatsoever.

### 3.4 STAGE I PRODUCT LABEL COMPARISON (STAGE1\_CAF\_PROPAGATION\_NOTEBOOK.IPYNB)

Run the notebook pointing at one vanilla v01p01 and one vanilla v04p04 stage1 ROOT file. This identifies product label changes that affect the NuGraph FCL sequences.

| Product Label                 | v01p01 NuGraph | v04p04 Vanilla | Significance                         |
|-------------------------------|----------------|----------------|--------------------------------------|
| pandoraGausNuGraphRecoCryoW/E | Present        | Absent         | Expected - NuGraph not yet in v04p04 |
| semantic                      | Present        | Absent         | Expected - NuGraph not yet in v04p04 |
| filter                        | Present        | Absent         | Expected - NuGraph not yet in v04p04 |

|                       |         |         |                                                     |
|-----------------------|---------|---------|-----------------------------------------------------|
| ngfilteredhitsCryoE/W | Present | Absent  | Expected - NuGraph not yet in v04p04                |
| combineHitsCryoW/E    | Absent  | Present | NEW 2D deco step - NuGraph FCLs must use this label |
| 400 / 600 / 300       | Absent  | Present | NEW WireCell tick grouping producers                |
| cluster3DCryoW/E      | Present | Present | Unchanged - safe to reference                       |
| pandoraGausCryoW/E    | Present | Present | Unchanged - safe to reference                       |
| SBNShowerGausCryoW/E  | Present | Present | Unchanged - safe to reference                       |

Critical finding: combineHitsCryoW/E replaces the v01p01 1D Gauss hit source.

In v04p04, hits feeding into Pandora come from the combineHits step. Any NuGraph module or FCL that references the old hit label must be updated to use combineHitsCryoW/E. This is a targeted string substitution in `nugraph_icarus.fcl` and `stage1_icarus_defs.fcl` - not a structural refactor.

#### A4. RECOMMENDED PORT SEQUENCE

---

Follow this order strictly. Building in the wrong sequence causes linker errors because downstream packages depend on headers that have not yet been updated.

| Step | Repo              | Risk        | Rationale                                                           |
|------|-------------------|-------------|---------------------------------------------------------------------|
| 1    | larpandoracontent | Low         | Most upstream; identical base; single header                        |
| 2    | larpandora        | Medium      | Depends on larpandoracontent; identical base; mostly additive C++   |
| 3    | sbnaobj           | Low-Medium  | Defines CAF types that sbncode fills - must come before sbncode     |
| 4    | icaruscode        | Medium-High | Stage1 FCL integration; independent of sbn* - can build in parallel |
| 5    | sbncode           | Medium-High | Depends on sbnaobj for types; needs stage1 output for testing       |

*Note: lardataobj and larrecodnn are skipped - their NuGraph changes were already merged into the base tags.*

## A5. TESTING CHECKPOINTS

| # | Checkpoint              | Command                                                                       | Expected Result            |
|---|-------------------------|-------------------------------------------------------------------------------|----------------------------|
| 1 | Build success           | <code>mrbl b 2&gt;&amp;1   tail -5</code>                                     | 0 errors, 'Build complete' |
| 2 | FCL dry run             | <code>lar --dry-run -c stage1_run2_icarus_NuGraphReco.fcl /dev/null</code>    | Exit 0, no ERROR lines     |
| 3 | Stage1 NuGraph products | <code>lar -c eventdump.fcl output.root 2&gt;&amp;1   grep NGMultiSlice</code> | Product entries visible    |
| 4 | CAF NuGraph branches    | Python: check branches with 'ng_' in name                                     | >0 branches found          |
| 5 | Score sanity            | Python: <code>ng_filt_pass_frac</code> mean/min/max                           | Values in [0, 1]           |

## A6. NUGRAPH2 PIPELINE DATA FLOW

---

This shows the full data flow within the NuGraph2-extended Stage 1 reconstruction.

| Module                        | Role / Output                                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input hits</b>             | combineHitsCryoW/E (v04p04) or gaushit (v01p01)                                                                                        |
| <b>NCCSlicesProducer</b>      | Identifies candidate neutrino slices from input hits                                                                                   |
| <b>NGMultiSliceInference</b>  | Runs NuGraph2 on each slice → ‘filter’ output (signal/noise score per hit) + ‘semantic’ output (MIP / HIP / Shower / Michel / Diffuse) |
| <b>ngfilteredhitsProducer</b> | Keeps only hits passing the filter score cut (configurable via NuGraphFilterCut param)                                                 |
| <b>pandoraGausNuGraphReco</b> | Second-pass Pandora reconstruction on filtered hits only                                                                               |
| <b>CAFMaker (sbncode)</b>     | FillSliceNuGraph() → rec.slc.ng_filt_pass_frac,<br>rec.slc.ng_plane[3]FillPFNuGraph() →<br>rec.slc.reco.pfp.ngscore.*                  |

## A7. QUICK REFERENCE: FEATURE BRANCH NAMES

---

I followed this naming convention when creating your branches so others can identify them:

| Repository        | Suggested Branch Name                                  |
|-------------------|--------------------------------------------------------|
| icaruscode        | feature/<username>_NuGraph2_Filter_v04p04              |
| sbnanaobj         | feature/<username>_NuGraph2_MoveVars_v04p04            |
| sbncode           | feature/<username>_CAFMaker_PandoraAfterNuGraph_v04p04 |
| larpandora        | feature/<username>_NuGraphInterface_v04p04             |
| larpandoracontent | feature/<username>_NuGraphInterface_v04p04             |

---

*This document was produced as part of the v10\_06\_00\_01p01 → v10\_06\_00\_04p04 NuGraph2 port effort (April 2026). The diagnostic notebooks referenced throughout should be rerun for any future port to a new base version, as the specific findings are version-specific.*



## B0. RECAP

### 0.1 WHAT CHANGED BETWEEN V01P01 AND V04P04

The key difference is the signal processing pipeline:

- ≡ v10\_06\_00\_01p01: 1D deconvolution. NuGraph hit label = pandoraGausCryoE/W
- ≡ v10\_06\_00\_04p04: 2D deconvolution (WireCell). NuGraph hit label = combineHitsCryoE/W

**WARNING:** The single most critical variable change: wherever NuGraph consumed the hit collection (HitLabel in nugraph\_icarus.fcl), the label changed from pandoraGausCryoE/W to combineHitsCryoE/W. All other pandoraGausCryoE/W references (PandoraLabel for slices and PFPs) remain correct and must NOT be changed.

### 0.2 DEPENDENCY VERSIONS

| Package           | Tag                               | Notes                        |
|-------------------|-----------------------------------|------------------------------|
| larpandoracontent | v04_15_01                         | Identical in both releases   |
| larpandora        | v10_00_19                         | Identical in both releases   |
| sbnanaobj         | v10_00_04 → v10_00_05             | Changed - must use v10_00_05 |
| sbncode           | v10_06_00_01 → v10_06_00_04       | Changed                      |
| icaruscode        | v10_06_00_01p01 → v10_06_00_04p04 | Changed                      |

### 0.3 DAILY STARTUP

```
ssh -KXY icarusgpvm05.fnal.gov
sh /exp/$(id -ng)/data/users/vito/podman/start_SL7dev_jsl.sh
source /cvmfs/icarus.opensciencegrid.org/products/icarus/setup_icarus.sh
htgettoken -a htvaultprod.fnal.gov -i icarus
```

– Separate terminal –

```
ssh icarusbuild02.fnal.gov
sh /exp/$(id -ng)/data/users/vito/podman/start_SL7dev_jsl.sh
source /cvmfs/icarus.opensciencegrid.org/products/icarus/setup_icarus.sh
setup icaruscode v10_06_00_04p04 -q e26:prof
cd /exp/icarus/app/users/sdey2/dev/dev_v10_06_00_04p04_nugraph
source localProducts*/setup && mrbslp
```

## Bl. LARPANDORACONTENT

---

|        |                                      |
|--------|--------------------------------------|
| Tag    | v04_15_01 (identical)                |
| Branch | feature/sdey_NuGraphInterface_v04p04 |
| Risk   | Low                                  |
| Result | BUILD SUCCESS                        |

### 1.1 STEPS

larpandoracontent is the same tag in both releases. The NuGraph interface files are pure additions with no conflicts.

≡ Check out at correct tag and create branch:

```
cd $MRB_SOURCE
```

```
mrbs g -t v04_15_01 larpandoracontent
```

```
cd larpandoracontent
```

```
git checkout -b feature/sdey_NuGraphInterface_v04p04
```

≡ Apply the patch - should be clean:

```
git apply --reject --whitespace=fix /path/to/full_diff_larpandoracontent.patch
```

```
find . -name '*.rej' | sort
```

≡ Build: mrbs i -j10

**INFO:** No errors encountered. Patch applied cleanly for this repository.

## B2. LARPANDORA

---

|        |                                      |
|--------|--------------------------------------|
| Tag    | v10_00_19 (identical)                |
| Branch | feature/sdey_NuGraphInterface_v04p04 |
| Risk   | Low                                  |
| Result | BUILD SUCCESS                        |

Same procedure as larpandoracontent. Same tag in both releases, pure additions.

```
cd $MRB_SOURCE && mrb g -t v10_00_19 larpandora
cd larpandora
git checkout -b feature/sdey_NuGraphInterface_v04p04
git apply --reject --whitespace=fix /path/to/full_diff_larpandora.patch
mrb i -j10
```

### B3. SBNANAOBJ

|        |                                       |
|--------|---------------------------------------|
| Tag    | v10_00_05 (was v10_00_04)             |
| Branch | feature/sdey_NuGraph2_MoveVars_v04p04 |
| Risk   | Medium                                |
| Result | BUILD SUCCESS                         |

#### 3.1 STEPS

- ⇒ **CRITICAL:** Check out v10\_00\_05, NOT v10\_00\_04:  
cd \$MRB\_SOURCE  
mrb g -t v10\_00\_05 sbnanaobj  
cd sbnanaobj  
git checkout -b feature/sdey\_NuGraph2\_MoveVars\_v04p04
- ⇒ Apply the patch:  
git apply --reject --whitespace=fix /path/to/full\_diff\_sbnanaobj.patch
- ⇒ Fix classes\_def.xml checksums (critical - see Section 3.2).
- ⇒ Build.

#### 3.2 THE CLASSES\_DEF.XML CHECKSUM PROBLEM

ROOT requires exact checksums for versioned classes. The checksums from v01p01 are WRONG for v04p04 because the data model changed in v10\_00\_05.

**WARNING:** Never copy checksums from v01p01 into v04p04. They will cause ROOT schema evolution errors at runtime even if the build succeeds. Always source checksums from the v10\_00\_05 baseline classes\_def.xml.

Correct v10\_00\_05 checksums:

| Class              | Correct Value                        |
|--------------------|--------------------------------------|
| SRNuGraphSliceInfo | ClassVersion=10, checksum=2711257956 |
| SRSlice            | ClassVersion=23, checksum=2614728424 |

Key rules:

- ⇒ checksum=0 is valid as a placeholder - ROOT computes the real value on first build
- ⇒ ClassVersion in classes\_def.xml must match ClassDef(ClassName, N) in the .h file
- ⇒ Preserve the full version history from v10\_00\_05 for SRSlice (v22 down to v1)
- ⇒ Phantom version entries from v01p01 intermediate history must be stripped

#### 3.3 NEW FIELDS IN V10\_00\_05

These fields are needed by sbncode FillReco.cxx and must be present:

```
// SRSlice.h:  
SRNuGraphSliceInfo ng_plane[3]; // per-plane NuGraph2 info
```

```
// SRNuGraphScore.h:
int mip_hits, hip_hits, shr_hits, mhl_hits, dif_hits;
int ng_vtx_hip_hits;
int unclustered_shr_hits;
enum NuGraphCategory { MIP, HIP, Shower, Michel, Diffuse, Unset };
```

## B4. ICARUSCODE

|        |                                     |
|--------|-------------------------------------|
| Tag    | v10_06_00_04p04                     |
| Branch | feature/sdey_NuGraph2_Filter_v04p04 |
| Risk   | High                                |
| Result | BUILD SUCCESS                       |

### 4.1 OVERVIEW

**icaruscode** requires the most manual work. The patch from v01p01 will have many rejects because FCL files changed significantly between releases. Always inspect the actual v04p04 file content before making edits.

### 4.2 CHECKOUT

```
cd $MRB_SOURCE
mrb g -t v10_06_00_04p04 icaruscode
cd icaruscode
git checkout -b feature/sdey_NuGraph2_Filter_v04p04
```

### 4.3 APPLY PATCH THEN RESET MODIFIED FILES

Apply with `--reject` to extract the pure additions (NuGraph/ directory, new FCLs), then reset the partially-modified existing files to a clean baseline:

```
git apply --reject --whitespace=fix /path/to/full_diff_icaruscode.patch
git checkout -- CMakeLists.txt fcl/caf/cafmaker_defs.fcl \
  fcl/reco/Definitions/stagel_icarus_defs.fcl \
  fcl/reco/Stagel/data/stagel_run2_1d_icarus.fcl \
  fcl/reco/Stagel/data/stagel_run2_icarus.fcl \
  fcl/reco/Stagel/mc/stagel_run2_1d_icarus_MC.fcl \
  fcl/reco/Stagel/mc/stagel_run2_icarus_MC.fcl \
  icaruscode/TPC/CMakeLists.txt \
  icaruscode/TPC/Tracking/ICARUSPandora/pandoramodules_icarus.fcl
find . -name '*.rej' -delete
```

**INFO:** The new untracked files (NuGraph/ directory, NuGraphReco FCLs) are correct and should be kept as-is.

### 4.4 MANUAL EDITS - 9 FILES

#### File 1: CMakeLists.txt (top-level)

Add `larrecodnn` after `larpandoracontent`. Check current line number first:

```
grep -n 'find_package' CMakeLists.txt | grep -A2 'larpandoracontent'
# Insert after larpandoracontent line (typically 74):
sed -i '74a find_package(larrecodnn REQUIRED )' CMakeLists.txt
grep -n 'larrecodnn' CMakeLists.txt # verify exactly once
```

#### File 2: icaruscode/TPC/CMakeLists.txt

Add NuGraph subdirectory after SPAna:

```
grep -n 'add_subdirectory' icaruscode/TPC/CMakeLists.txt
# Add after SPAna (typically line 11):
sed -i '11a add_subdirectory(NuGraph)' icaruscode/TPC/CMakeLists.txt
grep -n 'NuGraph' icaruscode/TPC/CMakeLists.txt # must be exactly once!
```

**WARNING: CRITICAL:** If NuGraph appears twice in this file you will get a CMake error 'binary directory already used'. Check with grep first and ensure NuGraph appears EXACTLY ONCE.

### File 3: pandoramodules\_icarus.fcl

Add 2 commented NuGraph lines after the closing brace of icarus\_basicpandora (check line ~23):

```
sed -n '20,28p' icaruscode/TPC/Tracking/ICARUSPandora/pandoramodules_icarus.fcl
# Manually add after closing } of icarus_basicpandora:
# NuGraph settings (enable when running NuGraph):
# icarus_basicpandora.ShouldCollectHitPredictions: true
```

### File 4: fcl/caf/cafmaker\_defs.fcl

Add all NuGraph CAF parameters:

```
grep -n 'TrackHitFillRREndCut' fcl/caf/cafmaker_defs.fcl # find line number
# Insert after that line (typically 318):
sed -i '318a cafmaker.NuGraphSlicesLabel: "NCCSlicesCryoE"\
ncafmaker.NuGraphFilterLabel: "filter"\
ncafmaker.NuGraphSemanticLabel: "semantic"\
ncafmaker.UsePandoraAfterNuGraph: true\
ncafmaker.NuGraphFilterCut: 0.5\
ncafmaker.NuGraphHIPTagWireDist: 10\
ncafmaker.NuGraphHIPTagTickDist: 50' fcl/caf/cafmaker_defs.fcl
grep -n 'NuGraph' fcl/caf/cafmaker_defs.fcl # verify 7 lines
```

### File 5: stage1\_icarus\_defs.fcl (the big one)

Requires 3 groups of additions. Use a Python script for reliability. Add:

⇒ Two #include lines after tpcpmtbarycentermatch\_config.fcl:

```
#include "pandoramodules_icarus.fcl"
#include "nugraph_icarus.fcl"
```

⇒ NuGraph producers in icarus\_stage1\_producers block (after tpcpmtbarycentermatchCryoW):

```
## NuGraph producers
nuslhitsCryoE/W, NuGraphCryoE/W, NCCSlicesCryoE/W,
NGMultiSliceCryoE/W, ngfilteredhitsCryoE/W,
pandoraGausNuGraphRecoCryoE/W: { @table::icarus_pandora
  HitFinderModuleLabel: ngfilteredhitsCryoE/W }
```

⇒ postPandora sequences after icarus\_pandora\_Gauss:

```
icarus_reco_postPandoraGausCryoW/E: [ NCCSlices, NGMultiSlice ]
icarus_reco_postPandoraGausCryoW/E_NuGraphReco: [ NCCSlices, NGMultiSlice,
  ngfilteredhits, pandoraGausNuGraphReco, pandoraTrack, pandoraKalman, SBNShower ]
```

```

icarus_postPandora_Gauss: [ postPandoraGausCryoE, postPandoraGausCryoW ]
icarus_postPandora_Gauss_NuGraphReco: [ ...NuGraphReco variants ]

```

#### Files 6-9: The 4 Stage1 Run FCLs

Insert @sequence::icarus\_postPandora\_Gauss after pandora/Gauss sequences, BEFORE icarus\_reco\_fm:

| File                                                                                                                                                                                               | Insertion Point                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| stage1_run2_icarus.fcl                                                                                                                                                                             | After @sequence::icarus_pandora_Gauss      |
| stage1_run2_1d_icarus.fcl                                                                                                                                                                          | After @sequence::icarus_pandora_Gauss      |
| stage1_run2_icarus_MC.fcl                                                                                                                                                                          | After @sequence::icarus_reco_Gauss2D_CryoW |
| stage1_run2_1d_icarus_MC.fcl                                                                                                                                                                       | After @sequence::icarus_reco_Gauss1D_CryoW |
| <b>WARNING:</b> For MC FCLs, postPandora_Gauss goes AFTER both CryoE and CryoW sequences. Final order: Gauss_CryoE, Gauss_CryoW, postPandora_Gauss, icarus_reco_fm. Never between CryoE and CryoW. |                                            |

```

# Verify all 4 FCLs after editing:
grep -n 'postPandora_Gauss\|pandora_Gauss\|Gauss2D_CryoW\|reco_fm' \
  fcl/reco/Stage1/data/stage1_run2_icarus.fcl \
  fcl/reco/Stage1/data/stage1_run2_1d_icarus.fcl \
  fcl/reco/Stage1/mc/stage1_run2_icarus_MC.fcl \
  fcl/reco/Stage1/mc/stage1_run2_1d_icarus_MC.fcl

```

#### 4.5 THE CRITICAL V04P04 VARIABLE FIX

After applying the patch, nugraph\_icarus.fcl has the old v01p01 hit label. Fix it:

```

grep -n 'HitLabel' icaruscode/TPC/NuGraph/nugraph_icarus.fcl
# Fix both labels:
sed -i 's/HitLabel: "pandoraGausCryoE"/HitLabel: "combineHitsCryoE"/' \
  icaruscode/TPC/NuGraph/nugraph_icarus.fcl
sed -i 's/HitLabel: "pandoraGausCryoW"/HitLabel: "combineHitsCryoW"/' \
  icaruscode/TPC/NuGraph/nugraph_icarus.fcl
# Verify:
grep -n 'HitLabel' icaruscode/TPC/NuGraph/nugraph_icarus.fcl

```

**NOTE:** PandoraLabel references (pandoraGausCryoE/W) in nugraph\_icarus.fcl and C++ module defaults are CORRECT - they refer to Pandora reco products (slices/PFPs), not the hit collection. Do NOT change them.

#### 4.6 COMMIT AND BUILD

```

git add -A
git commit -m 'Port NuGraph2 icaruscode changes from v01p01 to v04p04'
cd $MRB_BUILDDIR && mrb i -j10 2>&1 | tee build_icaruscode.log

```

#### 4.7 COMMON ERRORS

| Error                                             | Fix                                                                                           |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------|
| CMake: binary directory already used              | Duplicate add_subdirectory(NuGraph). Remove duplicate, delete build/icaruscode/, rerun mrb i. |
| CMake: subdirectory inclusion order               | Run mrb uc from \$MRB_SOURCE.                                                                 |
| ROOT schema evolution error                       | Wrong checksums in classes_def.xml. Use v10_00_05 checksums only.                             |
| FCL: unknown sequence<br>icarus_postPandora_Gauss | Check stage1_icarus_defs.fcl includes both pandoramodules_icarus.fcl and nugraph_icarus.fcl.  |
| FCL: unknown label combineHits*                   | Check stage1_icarus_defs.fcl has combineHitsCryoE/W producers defined.                        |

## B5. SBNCODE

|        |                                                  |
|--------|--------------------------------------------------|
| Tag    | v10_06_00_04                                     |
| Branch | feature/sdey_CAFMaker_PandoraAfterNuGraph_v04p04 |
| Risk   | Medium                                           |
| Result | BUILD SUCCESS                                    |

### 5.1 OVERVIEW

sbncode changes are in 4 files in sbncode/CAFMaker/. The v04p04 patch significantly refactors NuGraph data retrieval: switching from a pre-fetched slice hit map to per-slice art::FindOneP lookups, and adding plane-by-plane semantic variables.

| File               | Changes                                                                                                                    |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| CAFMakerParams.h   | Rename NuGraphSliceHitLabel -> NuGraphSlicesLabel. Add 4 new params.                                                       |
| CAFMaker_module.cc | Remove old slice hit map handles. Add nuGraphSlices vector. Replace FillSlice/PFPNuGraph calls with art::FindOneP lookups. |
| FillReco.cxx       | Remove ng_filter_cut global. Rewrite FillSliceNuGraph with plane-by-plane vars. Refactor FillPFPNuGraph.                   |
| FillReco.h         | Update function signatures.                                                                                                |

### 5.2 CHECKOUT AND APPLY

```
cd $MRB_SOURCE
mrb g -t v10_06_00_04 sbncode
cd sbncode
git checkout -b feature/sdey_CAFMaker_PandoraAfterNuGraph_v04p04
git apply --reject --whitespace=fix /path/to/full_diff_sbncode.patch
find . -name '*.rej' | sort # should be empty
```

**SUCCESS:** In the v04p04 port, all 4 sbncode files applied cleanly with zero rejects.

### 5.3 KEY API CHANGES (REFERENCE IF MANUAL EDITING NEEDED)

#### 5.3.1 CAFMakerParams.h - Renamed and New Params

```
// OLD: NuGraphSliceHitLabel
// NEW: NuGraphSlicesLabel
// NEW ADDITIONS:
Atom<bool> UsePandoraAfterNuGraph { Name(...), false }
```

```

Atom<float> NuGraphFilterCut      { Name(...), 0.5  }
Atom<float> NuGraphHIPTagWireDist { Name(...), 10   }
Atom<float> NuGraphHIPTagTickDist { Name(...), 50   }

```

### 5.3.2 FillSliceNuGraph - New Signature

```

// OLD: takes sliceHitsMap, uses global ng_filter_cut
void FillSliceNuGraph(inputHits, sliceHitsMap, ngFilter, ngSemantic, slice);

// NEW: loses sliceHitsMap, gains fmPFPPartHits + vertex + filter_cut
void FillSliceNuGraph(inputHits, ngFilter, ngSemantic,
    fmPFPPartHits, vtx_wire[3], vtx_tick[3],
    vtx_wire_dist, vtx_tick_dist, filter_cut, slice);

```

### 5.3.3 FillPFPNuGraph - New Signature

```

// OLD: takes sliceHitsMap + separate pfpHits
void FillPFPNuGraph(sliceHitsMap, ngFilter, ngSemantic, pfpHits, srpfp);

// NEW: takes pfpHits directly, adds filter_cut param
void FillPFPNuGraph(pfpHits, ngFilter, ngSemantic, filter_cut, srpfp);

```

### 5.3.4 Hit Lookup: std::find -> art::FindOneP

```

// OLD: global prefetch + std::find
GetByLabelIfExists(evt, NuGraphSliceHitLabel, handle);
auto it = std::find(sliceHitsMap.begin(), sliceHitsMap.end(), hit.key());

// NEW: per-slice art::FindOneP
art::FindOneP<anab::FeatureVector<1>> findOneFilter(
    slcHits, evt, NuGraphFilterLabel + slice_tag_suff + ":" + instance);
for (size_t i = 0; i < slcHits.size(); ++i)
    ng2_filter_vec.emplace_back(findOneFilter.at(i));

```

### 5.3.5 nuGraphSlices Logic

```

std::vector<art::Ptr<recob::Slice>> nuGraphSlices;
if (fParams.UsePandoraAfterNuGraph())
    nuGraphSlices = slices; // all slices
else
    nuGraphSlices = evt.getProduct<...>(NuGraphSlicesLabel + pandora_tag_suffix);

// Then guard all NuGraph filling:
if (std::find(nuGraphSlices.begin(), nuGraphSlices.end(), slice) !=
    nuGraphSlices.end()) {
    // fill NuGraph variables for this slice
}

```

## 5.4 UPDATE CAFMAKER\_DEFS.FCL IN ICARUSCODE

After patching sbncode, update icaruscode cafmaker\_defs.fcl with all NuGraph parameters:

```

# In fcl/caf/cafmaker_defs.fcl - full NuGraph block:

```

```

cafmaker.NuGraphSlicesLabel:      "NCCSlicesCryoE"
cafmaker.NuGraphFilterLabel:      "filter"
cafmaker.NuGraphSemanticLabel:    "semantic"
cafmaker.UsePandoraAfterNuGraph:  true
cafmaker.NuGraphFilterCut:        0.5
cafmaker.NuGraphHIPTagWireDist:   10
cafmaker.NuGraphHIPTagTickDist:   50

```

**WARNING:** NuGraphSliceHitLabel is gone. Using the old name will cause a fatal FHiCL validation error at runtime.

## 5.5 COMMIT AND BUILD

```

cd $MRB_SOURCE/sbncode
git add sbncode/CAFMaker/CAFMakerParams.h sbncode/CAFMaker/CAFMaker_module.cc \
      sbncode/CAFMaker/FillReco.cxx sbncode/CAFMaker/FillReco.h
git commit -m 'Port NuGraph2 CAFMaker changes from v01p01 to v04p04'

# Also amend icaruscode to include cafmaker_defs.fcl:
cd $MRB_SOURCE/icaruscode
git add fcl/caf/cafmaker_defs.fcl && git commit --amend --no-edit

# Run mrb uc to get correct build order, then build:
cd $MRB_SOURCE && mrb uc
mrbsetenv
cd $MRB_BUILDDIR && mrb i -j10 2>&1 | tee build_sbncode.log

```

## 5.6 COMMON ERRORS

| Error                                   | Fix                                                                                                                                                                                             |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CMake subdirectory order error          | Run mrb uc after adding sbncode. <b>Correct order:</b> larpandoracontent, larpandora, sbnanaobj, icaruscode, sbncode - just manually change this since mrb uc doesn't always do this correctly. |
| FHiCL: unknown key NuGraphSliceHitLabel | Update cafmaker_defs.fcl to NuGraphSlicesLabel.                                                                                                                                                 |
| Compile: ng_filter_cut undeclared       | Remove the old global from FillReco.cxx - it is now a function parameter.                                                                                                                       |
| art exception on FindOneP               | Check NuGraphFilterLabel and NuGraphSemanticLabel are set correctly in FCL.                                                                                                                     |

## B6. FINAL CHECKLIST

### 6.1 BUILD ORDER

| Order                | Command                                        |
|----------------------|------------------------------------------------|
| 1. larpandoracontent | mrbs g -t v04_15_01 larpandoracontent          |
| 2. larpandora        | mrbs g -t v10_00_19 larpandora                 |
| 3. sbnanaobj         | mrbs g -t v10_00_05 sbnanaobj (NOT v10_00_04!) |
| 4. icaruscode        | mrbs g -t v10_06_00_04p04 icaruscode           |
| 5. sbncode           | mrbs g -t v10_06_00_04 sbncode                 |

### 6.2 PRE-BUILD CHECKLIST

- ≡ icaruscode/TPC/CMakeLists.txt has add\_subdirectory(NuGraph) exactly ONCE
- ≡ nugraph\_icarus.fcl: HitLabel = combineHitsCryoE/W (not pandoraGausCryoE/W)
- ≡ cafmaker\_defs.fcl: uses NuGraphSlicesLabel (not NuGraphSliceHitLabel)
- ≡ cafmaker\_defs.fcl: has all 7 NuGraph parameters
- ≡ stage1\_icarus\_defs.fcl: includes pandoramodules\_icarus.fcl AND nugraph\_icarus.fcl
- ≡ All 4 stage1 run FCLs: have @sequence::icarus\_postPandora\_Gauss in correct position
- ≡ sbnanaobj classes\_def.xml: uses v10\_00\_05 checksums
- ≡ mrbs uc run after adding all 5 packages (though typically it still gives you a messed up order so put it in manually or just follow my order)

### 6.3 VARIABLE CHANGE SUMMARY

| Item                           | v01p01                          | v04p04                        |
|--------------------------------|---------------------------------|-------------------------------|
| nugraph_icarus.fcl<br>HitLabel | pandoraGausCryoE/W              | combineHitsCryoE/W            |
| CAFMakerParams param name      | NuGraphSliceHitLabel            | NuGraphSlicesLabel            |
| New param                      | (absent)                        | UsePandoraAfterNuGraph = true |
| New param                      | (absent)                        | NuGraphFilterCut = 0.5        |
| New param                      | (absent)                        | NuGraphHIPTagWireDist = 10    |
| New param                      | (absent)                        | NuGraphHIPTagTickDist = 50    |
| FillReco.cxx global            | const float ng_filter_cut = 0.5 | Removed - function param      |
| sbnanaobj tag                  | v10_00_04                       | v10_00_05                     |

## 6.4 BRANCH NAMES

| Repository        | Branch                                           |
|-------------------|--------------------------------------------------|
| larpandoracontent | feature/sdey_NuGraphInterface_v04p04             |
| larpandora        | feature/sdey_NuGraphInterface_v04p04             |
| sbnanaobj         | feature/sdey_NuGraph2_MoveVars_v04p04            |
| icaruscode        | feature/sdey_NuGraph2_Filter_v04p04              |
| sbncode           | feature/sdey_CAFMaker_PandoraAfterNuGraph_v04p04 |

**SUCCESS:** All 5 repositories successfully ported and built for v10\_06\_00\_04p04. Next step: run a test job with `stage1_run2_icarus_NuGraphReco.fcl` to verify runtime correctness of the full NuGraph2 pipeline.



## C. POST-BUILD RUNTIME DEBUGGING AND FINAL FIXES

### C0. CONTEXT & STARTING POINT

---

This section covers the runtime debugging phase that followed the successful build of all five ported repositories. At the start of Part C, the build completes without errors and art can load the FCL. The issues from this point forward are runtime crashes and incorrect data product associations, only visible when lar is actually executed.

**Parts A and B** of this guide covered the port itself: which files to change, in which repos, and in what order. **Part C** covers what happened next, how it was diagnosed, and exactly what was fixed.

## C1. FIRST RUNTIME TEST — SEGFAULT (HEAP CORRUPTION)

### C1.1 THE RUN

After a successful build, the standard run command was:

```
lar -c stage1_run2_icarus_overlay_NuGraphReco.fcl -n 1 \  
-s /pnfs/sbn/data/sbn_fd/poms_production/mc/2025A_ICARUS_Overlays_BNB_RUN2/\  
September/v10_06_00_04p04/stage0/00/02/\  
stage0_run2_wcdnn_icarus_overlay_26910969_7314-...root \  
--process-name redo -o stage1-nugraph-test.root \  
--trace > stage1_nugraph.log 2>&1
```

The `--trace` flag prints each module as it starts and finishes — the fastest way to find where a crash occurs.

The job crashed with a segfault. The `--trace` output showed:

```
+++++++ module for event: NGMultiSliceCryoE  
+++++++ finished for event: NGMultiSliceCryoE    <- NuGraph inference SUCCEEDS  
+++++++ module for event: ngfilteredhitsCryoE    <- log cuts off here, core  
dump
```

The job died at the start of `ngfilteredhitsCryoE` (`ICARUSFilteredNuSliceHitsProducer`). `NuGraph` inference itself completed successfully.

### C1.2 DIAGNOSTIC APPROACH — THREE TOOLS

Three complementary tools were used to diagnose the crash. Each reveals a different layer of the problem.

#### Tool 1: GDB — Where Did It Crash?

GDB catches the crash and prints a backtrace showing the call stack at the moment of the segfault. Before running, unset Python environment variables that cause GDB's embedded Python to fail:

```
unset PYTHONPATH PYTHONHOME  
  
gdb -ex "set logging file gdb_nugraph_bt.log" \  
-ex "set logging on" \  
-ex "run" \  
-ex "bt" \  
-ex "bt full" \  
-ex "quit" \  
--args lar -c stage1_run2_icarus_overlay_NuGraphReco.fcl -n 1 \  
-s <input_file> --process-name redo -o /dev/null 2>&1
```

Key GDB backtrace frames:

```
#0  _int_malloc                                <- heap allocator corrupted  
#1  malloc  
#2  operator new  
#3  art::detail::IPRHelper<recob::Slice, recob::Hit...>
```

```
<- FindManyP<Hit> construction inside ICARUSNuGraphInference_module.so
#4 ICARUSFilteredNuSliceHitsProducer::produce
```

The crash is in `_int_malloc` — the heap allocator itself. This is a classic heap corruption pattern: the crash site (malloc) and the bug site (the bad write) are different locations. The actual out-of-bounds write happened upstream, and `ICARUSFilteredNuSliceHitsProducer` is simply the first module to call malloc after the corruption.

## Tool 2: Valgrind — Where Was the Bad Write?

Valgrind intercepts every memory read and write at the CPU level, catching the invalid write at the exact moment it happens. The tradeoff is **10-100x** slower execution. Always run in the background with `&`.

```
unset PYTHONPATH PYTHONHOME
setup valgrind v3_12_0 -q "Linux64bit+3.10-2.17"

valgrind --tool=memcheck \
  --error-limit=no \
  --track-origins=yes \
  --num-callers=30 \
  lar -c stage1_run2_icarus_overlay_NuGraphReco.fcl -n 1 \
  -s <input_file> --process-name redo -o /dev/null \
  > valgrind_nugraph.log 2>&1 &

# Monitor progress:
tail -f valgrind_nugraph.log
```

| Flag                             | Meaning                                                                   |
|----------------------------------|---------------------------------------------------------------------------|
| <code>--tool=memcheck</code>     | Use the memory checking tool (default but explicit)                       |
| <code>--error-limit=no</code>    | Do not stop reporting after 1000 errors                                   |
| <code>--track-origins=yes</code> | Report where uninitialised values came from (slower but more informative) |
| <code>--num-callers=30</code>    | Show 30 stack frames per error instead of default 12                      |
| <code>&amp;</code>               | Run in background, freeing the terminal                                   |

Valgrind confirmed three invalid reads inside `HitMerger` (`makeWireAssns`, `makeRawDigitAssns`, `makeChanROIAssns`) and zero invalid reads inside any NuGraph code. The `HitMerger` errors were identical in both the NuGraph run and a vanilla v04p04 run - confirming a pre-existing bug in base icaruscode, not something introduced by the port.

## Tool 3: Debug Build — For Future Reference

GDB output from a prof (optimised) build often shows variables as `<optimised out>`. A debug build compiles with `-O0 -g`, giving full symbol visibility. To add a debug build alongside the existing prof build:

```
# From the top-level dev area directory:
cd /path/to/dev_v10_06_00_04p04_nugraph

# -T names the new build directory, -f reuses existing srcs/
mrb newDev -T debug_build -q e26:debug -f

# Source the debug environment and build:
```

```
source localProducts*debug_build*/setup
mrbsetenv
mrb i -j8 2>&1 | tee build_debug.log
```

**WARNING:** A debug build is significantly larger and slower than a prof build. Only use it when you need to inspect local variable values in GDB. For this port, the prof build combined with valgrind and printouts was sufficient.

## C2. FIXES APPLIED

---

### C2.1 FIX: HITMERGER VECTOR REALLOCATION BUG

**Note:** This bug exists in vanilla v04p04 and is not introduced by the NuGraph port. It should be submitted as a PR to base icaruscode.

HitMerger is a v04p04-specific module (absent in v01p01) that merges per-TPC hit collections from WireCell 2D deconvolution into a single collection per cryostat.

```
gaushit2dTPCEW  -->|
                  |--> HitMerger --> combineHitsCryoE
gaushit2dTPCEE  -->|
```

**File:** icaruscode/TPC/SignalProcessing/HitFinder/HitMerger\_module.cc

Root cause: HitMerger::produce() stores raw recob::Hit\* pointers as keys in recobHitToPtrMap. With two input labels (gaushit2dTPCEW and gaushit2dTPCEE), emplace\_back on the second pass triggers reallocation of outputHitPtrVec, moving the vector to a new memory location. All raw pointers stored from the first pass are now dangling. The makeWireAssns, makeRawDigitAssns, and makeChanROIAssns functions then dereference those dangling pointers - reading from freed memory.

Fix: pre-count all hits and call reserve() before the loop so the vector never reallocates.

```
// Sparshita Dey 2026 -- pre-count to prevent reallocation invalidating raw
pointers
size_t totalHits = 0;
for (const auto& inputTag : HitMergerfHitProducerLabelVec) {
    art::Handle<std::vector<recob::Hit>> hitHandle;
    evt.getByLabel(inputTag, hitHandle);
    if (hitHandle.isValid()) totalHits += hitHandle->size();
}
outputHitPtrVec->reserve(totalHits);
```

After applying this fix and rebuilding, valgrind confirmed the HitMerger errors were gone.

### C2.2 FIX: NGMULTISLICECRYOW WRONG HITINPUT (FCL BUG)

**File:** icaruscode/TPC/NuGraph/nugraph\_icarus.fcl

NGMultiSliceCryoW was defined as @local::NGMultiSliceCryoE and inherited hitInput: nuslhitsCryoE for its loader and decoder tools - feeding the west cryostat NuGraph with east cryostat hits.

Fix: add explicit overrides after the NGMultiSliceCryoW block in all three FCL copies (srcs/, localProducts\*/, build\_slf7\*/):

```
NGMultiSliceCryoW.LoaderTool.hitInput: "nuslhitsCryoW"
NGMultiSliceCryoW.LoaderTool.spsInput: "nuslhitsCryoW"
```

**WARNING:** Remember: `lar` reads from `localProducts*/`, not `srcs/`. All three copies of every FCL must be edited for the change to take effect at runtime.

## C2.3 FIX: SWITCH PANDORA TO CLUSTER3DCRYOE (ARCHITECTURE FIX)

### C2.3a Why This Was Needed

In the initial port, Pandora read hits from `combineHitsCryoE` (produced by `HitMerger` from `gaushit2dTPCEW/EE`). NuGraph's decoder tools, however, read `SpacePoints` from `cluster3DCryoE` (produced from `gaushitPT2dTPCEW/EE` - the pass-through hits). These are entirely separate hit collections with different `art::Ptr` keys.

The consequence: when NuGraph's `FilterDecoder::writeToEvent` used `idsmap` entries (`art::Ptr` keys into `cluster3DCryoE`) to index into `filtcol` (sized by the count of hits from `combineHitsCryoE`), it wrote out of bounds. This is the heap corruption that GDB caught in `_int_malloc`.

Giuseppe confirmed the correct fix: NuGraph requires a single consistent hit source feeding Pandora, the loader, and the decoder. The right choice is `cluster3DCryoE`, which is the source the NuGraph decoder tools already expected.

### C2.3b The Fix

**File:** `icaruscode/fcl/reco/Definitions/stagel_icarus_defs.fcl`, lines 426 and 434

```
# Change from:
icarus_stagel_producers.pandoraGausCryoW.HitFinderModuleLabel:
"combineHitsCryoW"
icarus_stagel_producers.pandoraGausCryoE.HitFinderModuleLabel:
"combineHitsCryoE"

# Change to:
icarus_stagel_producers.pandoraGausCryoW.HitFinderModuleLabel: "cluster3DCryoW"
icarus_stagel_producers.pandoraGausCryoE.HitFinderModuleLabel: "cluster3DCryoE"
```

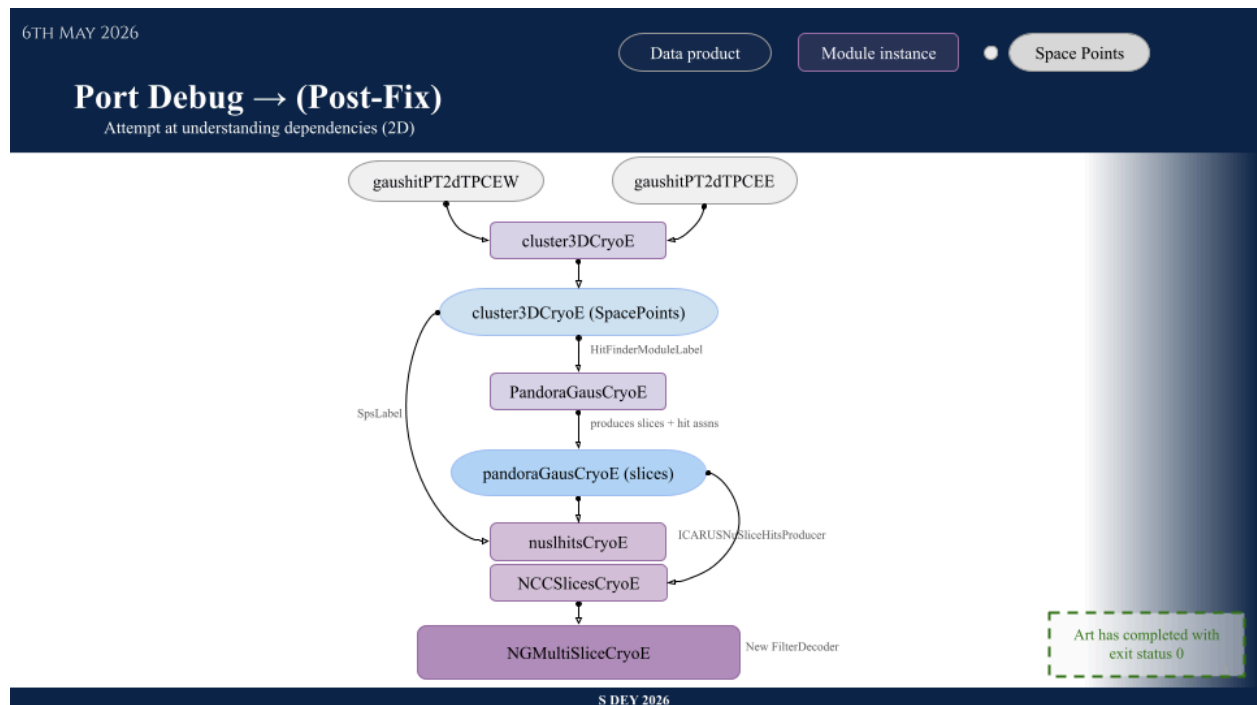
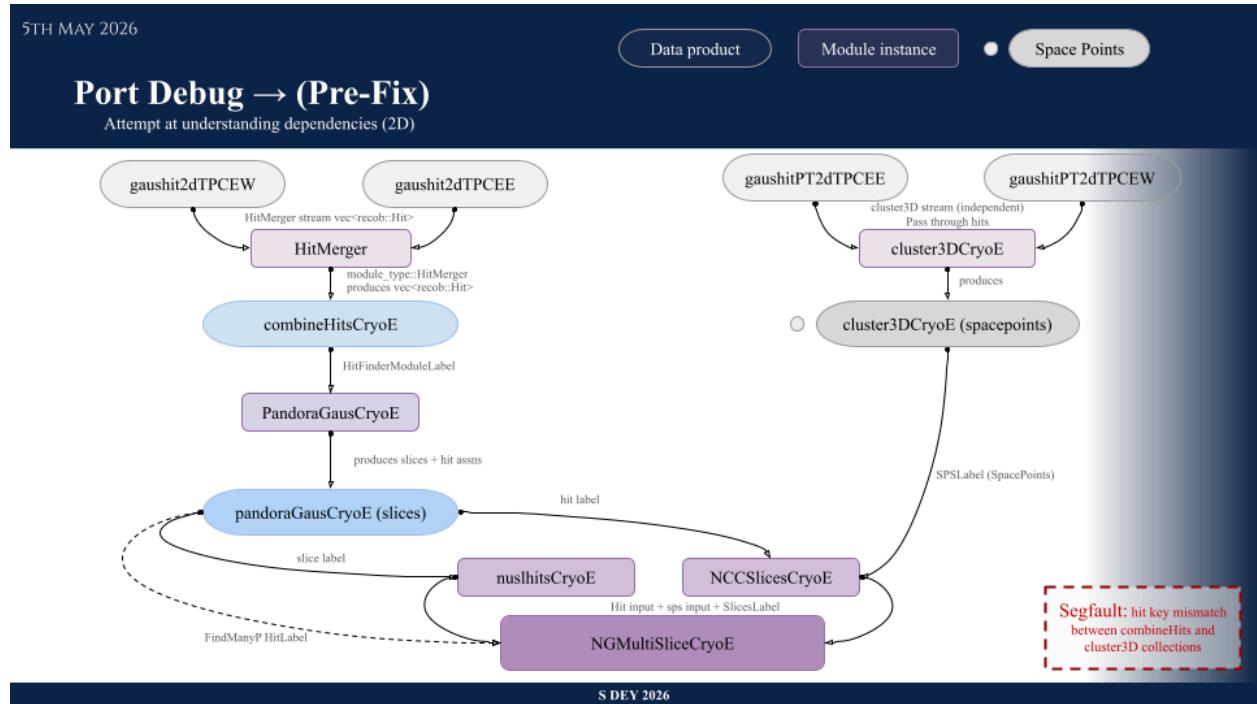
Apply using `sed` across all three FCL copies:

```
for fcl in \
  srcs/icaruscode/fcl/reco/Definitions/stagel_icarus_defs.fcl \
  localProducts*/icaruscode/*/job/stagel_icarus_defs.fcl \
  build_slf7.x86_64/icaruscode/job/stagel_icarus_defs.fcl; do
  sed -i
  's/pandoraGausCryoW.HitFinderModuleLabel:.*"combineHitsCryoW"/pandoraGausCryoW.
  HitFinderModuleLabel: "cluster3DCryoW"/' $fcl
  sed -i
  's/pandoraGausCryoE.HitFinderModuleLabel:.*"combineHitsCryoE"/pandoraGausCryoE.
  HitFinderModuleLabel: "cluster3DCryoE"/' $fcl
done

# Verify (should show cluster3DCryoE/W in all 6 lines):
grep -n 'HitFinderModuleLabel' \
  srcs/icaruscode/fcl/reco/Definitions/stagel_icarus_defs.fcl \
  localProducts*/icaruscode/*/job/stagel_icarus_defs.fcl \
  build_slf7.x86_64/icaruscode/job/stagel_icarus_defs.fcl
```

**Note:** `HitMerger` still runs and produces `combineHitsCryoE/W`, but these collections are no longer consumed by Pandora or NuGraph. They are unused downstream.

## C2.3c Pipeline: Before and After



Note: See accompanying diagrams showing the before/after pipeline data flow.

## C2.4 FIX: LARDATAOBJ AND LARRECODNN FROM LEONARDO'S BRANCH

### C2.4a The FilterDecoder Bug

Even with the `cluster3D` switch, the pipeline crashed in `FilterDecoder::writeToEvent`. The version of `FilterDecoder_tool.cc` on `cvmfs (larrecodnn v10_01_10)` had an indexing bug:

```
// filtcol is sized by COUNT of hits in slices (e.g. 200):
size_t size = 0;
for (auto& v : idsmmap) size += v.size();
auto filtcol = make_unique<vector<FeatureVector<1>>>(size, ...);

// But idx is a raw art::Ptr KEY into the full hit collection (e.g. 4532):
size_t idx = idsmmap[p][i];
(*filtcol)[idx] = FeatureVector<1>(input); // CRASH if 4532 > 200
```

The raw `art::Ptr` key (which indexes into the full hit collection) was used as a direct positional index into `filtcol` (sized only by the number of hits in slices). If any hit has a key larger than the count of processed hits, this writes out of bounds.

### C2.4b The Fix — Leonardo's Branch

Leonardo Lena's `feature/nugraph-multislice` branch of `larrecodnn` contains a corrected `FilterDecoder_tool.cc` that builds a sorted key list and uses `std::find + std::distance` to convert raw keys to correct positional indices. It also introduces `art::Assns<FeatureVector<1>, recob::Hit>` for proper hit-score associations. Check out both `lardataobj` and `larrecodnn` from Leonardo's branch:

```
cd $MRB_SOURCE

# lardataobj
mrb g -t v10_01_00 lardataobj
cd lardataobj
git remote add llena https://github.com/leonardo-lena/lardataobj.git
git fetch llena
git checkout -B feature-nugraph-multislice llena/feature/nugraph-multislice
cd ..

# larrecodnn
mrb g -t v10_01_10 larrecodnn
cd larrecodnn
git remote add llena https://github.com/leonardo-lena/larrecodnn.git
git fetch llena
git checkout -B feature-nugraph-multislice llena/feature/nugraph-multislice
cd ..

# Rebuild:
cd $MRB_BUILDDIR
mrbsetenv
mrb i -j8 2>&1 | tee build_all.log
```

**WARNING:** `mrb uc` does not reliably produce the correct build order when `lardataobj` and `larrecodnn` are added (to fix this would mean editing up dependencies for all packages → not worth it! Edit `CMakeLists.txt` manually. Correct order: `larpandoracontent`, `lardataobj`, `larrecodnn`, `larpandora`, `sbnanaobj`, `icaruscode`, `sbncode`. The build will either stall or you will get dependency errors if the order is wrong - if this happens just correct it according to the error message, then `mrbsetenv`, then rebuild (`mrb i -j20`).

### C3. FIRST SUCCESSFUL END-TO-END RUN

**SUCCESS:** Art has completed and will exit with status 0. Events total = 1 passed = 1 failed = 0.

After all four fixes were applied and the build completed, the full NuGraph2 pipeline ran successfully for the first time on v04p04. All modules completed with 0 errors:

```
TrigReport Events total = 1 passed = 1 failed = 0
TimeReport CPU = 34.797811 Real = 44.653458
MemReport VmPeak = 4620.84 VmHWM = 2660.17
Art has completed and will exit with status 0.
```

#### C4. NUGRAPH2 HIT FILTER STATISTICS

Run on 5 MC overlay events (BNB RUN2, v10\_06\_00\_04p04 Stage 0 input). Filter score cut = 0.5. Hit counts reported by ICARUSFilteredNuSliceHitsProducer.

```
grep "Number of hits" stage1_nugraph_5events.log
```

| Event          | Cryostat | Hits before  | 1D model after | 1D % kept    | 2D model after | 2D % kept    |
|----------------|----------|--------------|----------------|--------------|----------------|--------------|
| 1              | CryoE    | 2165         | 1624           | 75.0%        | 1716           | 79.3%        |
| 1              | CryoW    | 2961         | 2376           | 80.2%        | 2522           | 85.2%        |
| 2              | CryoE    | 2793         | 2327           | 83.3%        | 2458           | 88.0%        |
| 2              | CryoW    | 3629         | 3073           | 84.7%        | 2168           | 59.7%        |
| 3              | CryoE    | 4804         | 3773           | 78.5%        | 3858           | 80.3%        |
| 3              | CryoW    | 3989         | 3368           | 84.4%        | 3435           | 86.1%        |
| 4              | CryoE    | 860          | 487            | 56.6%        | 554            | 64.4%        |
| 4              | CryoW    | 505          | 224            | 44.4%        | 352            | 69.7%        |
| 5              | CryoE    | 2983         | 1940           | 65.0%        | 2530           | 84.8%        |
| 5              | CryoW    | 2568         | 2030           | 79.0%        | 2177           | 84.8%        |
| <b>Overall</b> |          | <b>27257</b> | <b>21222</b>   | <b>77.9%</b> | <b>21770</b>   | <b>79.9%</b> |

**Note:** Event 2 CryoW is an outlier at 59.7% kept with the 2D model (vs 84.7% with the 1D model). This warrants further study on more events.

**Note:** The 1D model column is shown for reference only. It should not be used with 2D deconvolution inputs. See Section C5.

## C5. NUGRAPH2 MODEL

NuGraph2 uses a PyTorch model file (.pt) for inference, loaded at runtime via `FW_SEARCH_PATH`. No rebuild is needed to switch models. The `modelFileName` parameter in `nugraph_icarus.fcl` controls which file is loaded.

|                 | v10_06_00_01p01 (1D)              | v10_06_00_04p04 (2D)              |
|-----------------|-----------------------------------|-----------------------------------|
| Model file      | model_mpvmpmr_bnb_numu_cos.pt     | model_mpvmpmr_1201.pt             |
| Location        | cvmfs icarus_data/NuGraph/        | Local (not yet on cvmfs)          |
| Training sample | BNB numu + cosmics (1D hits)      | MPVMMPR (2D WireCell hits)        |
| Semantic labels | MIP, HIP, Shower, Michel, Diffuse | MIP, HIP, Shower, Michel, Diffuse |

To use the 2D model, add its directory to `FW_SEARCH_PATH` before running:

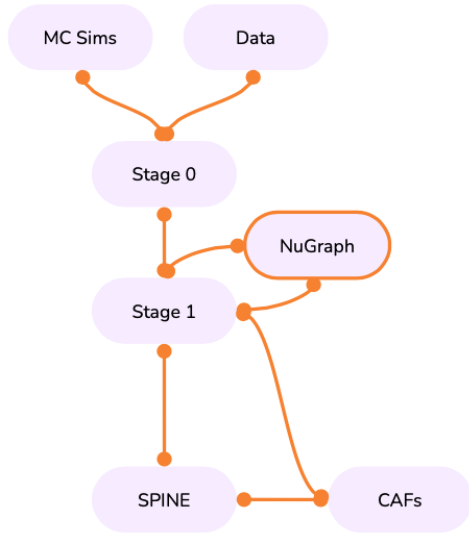
```
export FW_SEARCH_PATH=/path/to/model/directory:$FW_SEARCH_PATH
```

**WARNING:** The 2D model (model\_mpvmpmr\_1201.pt) is not yet on `cvmfs`. It must be present in a directory on `FW_SEARCH_PATH` at runtime. This should be submitted to `icarus_data` for production use.

## C6. CHANGE SUMMARY

| Fix                           | File                       | Type                | Notes                                                                          |
|-------------------------------|----------------------------|---------------------|--------------------------------------------------------------------------------|
| HitMerger<br>reserve()        | HitMerger_module.cc        | C++ — rebuild       | Pre-existing bug in base v04p04.<br>PR pending.                                |
| NGMultiSliceCryoW<br>hitInput | nugraph_icarus.fcl         | FCL only — 3 copies | CryoW inherited CryoE loader<br>inputs.                                        |
| Pandora →<br>cluster3DCryoE   | stage1_icarus_defs.<br>fcl | FCL only — 3 copies | Makes all hit sources consistent.                                              |
| New FilterDecoder             | larrecodnn (Ilena branch)  | Checkout + rebuild  | Fixes art::Ptr key →<br>positional index mismatch.                             |
| lardataobj                    | lardataobj (Ilena branch)  | Checkout + rebuild  | Class definitions for NuGraph<br>multi-slice data products.                    |
| 2D model                      | nugraph_icarus.fcl         | FCL only — 3 copies | Change modelFileName to<br>model_mvmpmr_1201.pt.<br>Add dir to FW_SEARCH_PATH. |





### A.1 WHAT IS NUGRAPH2?

NuGraph2 is a Graph Neural Network (GNN) designed for low-level reconstruction of neutrino interactions in liquid argon time projection chamber (LArTPC) detectors. It operates at the individual TPC hit level, processing the raw geometry of hits within a Pandora slice and assigning each hit two scores:

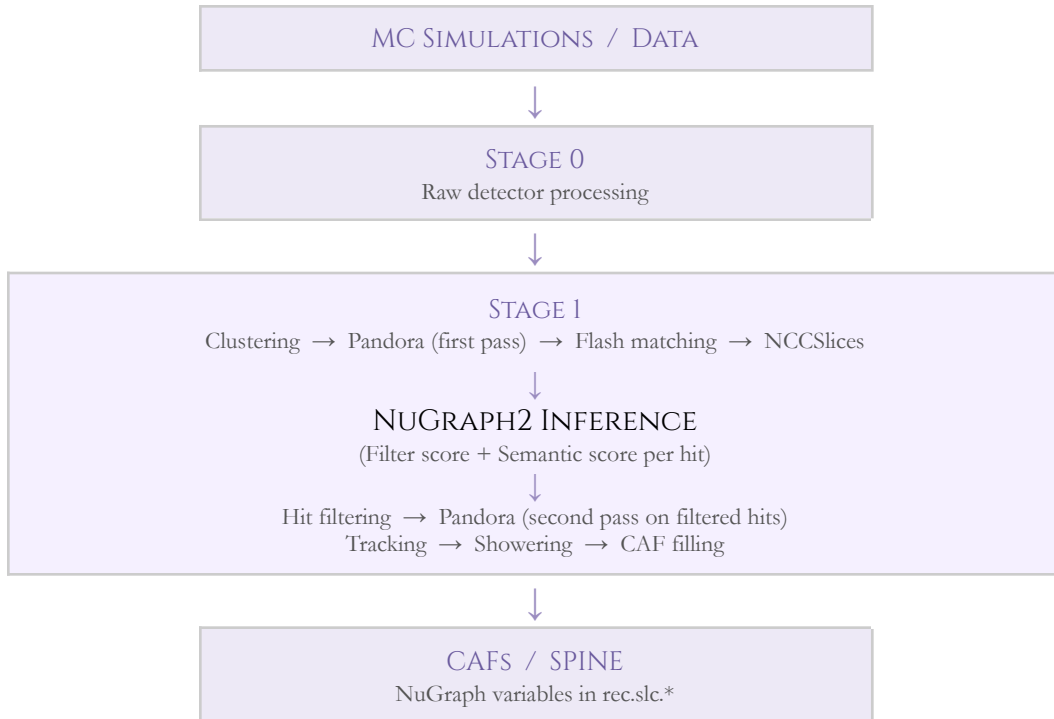
≡ A filter score (binary: signal vs background) used to remove noise and cosmic hits before a second-pass Pandora reconstruction.

≡ A semantic score (multi-class): **MIP** (minimum ionising particle), **HIP** (highly ionising particle), **Shower**, **Michel**, **Diffuse**.

NuGraph2 enhances Pandora reconstruction by providing hit-level physics context that standard Pandora algorithms cannot access. By running GNN inference over the full hit graph of a neutrino candidate slice, it captures long-range geometric relationships between hits across all three wire planes simultaneously.

### A.2 HOW NUGRAPH2 FITS INTO THE PIPELINE

NuGraph2 runs inside the Stage 1 reconstruction as an intermediate step between the first-pass Pandora reconstruction and the second-pass Pandora reconstruction on filtered hits. The full chain is:



### A.3 WHY NUGRAPH2 IMPROVES RECONSTRUCTION EFFICIENCY

Standard Pandora slice reconstruction is limited to local geometric heuristics within each wire plane. NuGraph2 overcomes this by constructing a graph over all hits in a neutrino candidate slice, connecting hits across planes via shared SpacePoints, and running message-passing inference to propagate physics context globally.

- ≡ The filter decoder removes background and cosmic-ray hits before the second-pass Pandora sees them, reducing fragmentation and improving particle flow object (PFP) quality.
- ≡ The semantic decoder provides hit-level particle type classification (MIP/HIP/Shower/Michel/Diffuse) which can be used downstream for particle identification and selection.
- ≡ By running on neutrino candidate slices only (selected by NCCSlicesProducer after flash matching), NuGraph2 processes only the relevant portion of each event, keeping runtime manageable.

**Note:** NuGraph2 runs after Pandora slicing and flash matching, and before the second-pass Pandora reconstruction on filtered hits. It does not replace Pandora — it enhances it by providing hit-level physics scores that inform the second-pass reconstruction.

#### A.4 CAF VARIABLES PRODUCED

The following NuGraph2 variables are written to the CAF by sbncode:

| CAF Variable                              | Description                                                    |
|-------------------------------------------|----------------------------------------------------------------|
| <code>rec.slc.ng_filt_pass_frac</code>    | Fraction of hits in slice passing the NuGraph filter score cut |
| <code>rec.slc.ng_plane[p].mip_hits</code> | Number of MIP-tagged hits per plane <i>p</i>                   |
| <code>rec.slc.ng_plane[p].hip_hits</code> | Number of HIP-tagged hits per plane <i>p</i>                   |
| <code>rec.slc.ng_plane[p].shr_hits</code> | Number of Shower-tagged hits per plane <i>p</i>                |
| <code>rec.slc.ng_plane[p].mhl_hits</code> | Number of Michel-tagged hits per plane <i>p</i>                |
| <code>rec.slc.ng_plane[p].dif_hits</code> | Number of Diffuse-tagged hits per plane <i>p</i>               |
| <code>rec.slc.reco.pfp.ngscore.*</code>   | Per-PFP NuGraph semantic scores                                |

## APPENDIX B | GREP CHEATSHEET

grep searches for patterns in files or command output. It is the fastest first diagnostic tool for any log file from a lar job.

### B.1 BASIC SYNTAX

```
grep [OPTIONS] PATTERN FILE
grep [OPTIONS] PATTERN FILE1 FILE2 ...
command | grep [OPTIONS] PATTERN # pipe from another command
```

### B.2 KEY OPTIONS

| Option         | Effect                                                                    |
|----------------|---------------------------------------------------------------------------|
| -i             | Case-insensitive match                                                    |
| -n             | Print line numbers alongside matches                                      |
| -v             | Invert match -- print lines that do NOT match                             |
| -r             | Recursive -- search all files in a directory tree                         |
| -l             | Print only filenames of files containing a match (not the matching lines) |
| -c             | Print only a count of matching lines per file                             |
| -A N           | Print N lines After each match (context below)                            |
| -B N           | Print N lines Before each match (context above)                           |
| -C N           | Print N lines of Context around each match (both above and below)         |
| -E             | Extended regex -- enables  , +, ?, () without escaping                    |
| -w             | Match whole words only                                                    |
| -m N           | Stop after N matches                                                      |
| --include=*.cc | Only search files matching a glob pattern (use with -r)                   |

### B.3 COMMON PATTERNS FOR LAR JOBS

Find errors and crashes in a log

```
grep -n 'error\|Error\|Abort\|Segmentation\|SIGSEGV\|Fatal\|MSG-e' job.log |
head -30
```

Track module execution with --trace

```
grep -n 'module for event\|finished for event' job.log | tail -30
```

Search across multiple files (3 FCL copies)

```
grep -n 'HitLabel' \
  srcs/icaruscode/icaruscode/TPC/NuGraph/nugraph_icarus.fcl \
  localProducts*/icaruscode/*/job/nugraph_icarus.fcl \
  build_slf7.x86_64/icaruscode/job/nugraph_icarus.fcl
```

Find a string recursively across all source files

```
grep -rn 'HitFinderModuleLabel' srcs/icaruscode/fcl/ | head -20
```

OR pattern -- multiple search terms

```
grep -n 'Invalid read\|Invalid write\|HitMerger\|FlashPredict' valgrind.log |  
head -40
```

Exclude lines matching a pattern

```
# Show all map lines that are not 7f... or ff... (memory map cleanup):  
grep -v '^7f\|^ff' stage1.log | tail -50
```

Count hits before and after NuGraph filter

```
grep 'Number of hits' stage1_5events.log
```

Pipe grep into grep (AND logic)

```
# Lines containing both 'Invalid' and 'HitMerger':  
grep 'Invalid' valgrind.log | grep 'HitMerger'
```

## B.4 USEFUL ONE-LINERS

Show only the last N lines of grep output

```
grep 'pattern' file.log | tail -20
```

Show matching lines with filename (multi-file search)

```
grep -Hn 'combineHits' srcs/icaruscode/fcl/**/*.fcl
```

Extract a block of lines around a match

```
grep -n -A5 -B2 'ICARUSFilteredNuSliceHitsProducer' stage1.log | head -40
```

Check all three FCL copies for a value

```
for f in srcs/icaruscode/icaruscode/TPC/NuGraph/nugraph_icarus.fcl \  
localProducts*/icaruscode/*/job/nugraph_icarus.fcl \  
build_slf7.x86_64/icaruscode/job/nugraph_icarus.fcl; do
```

```
    echo "=== $f ==="  
    grep 'HitLabel' $f  
done
```

## APPENDIX C | GDB CHEATSHEET

GDB (GNU Debugger) attaches to a running or crashed process and lets you inspect the call stack, variables, and memory at the moment of a crash. For lar jobs, it is used to find where a segfault occurred and what the call stack looked like.

### C.1 SETUP FOR LAR JOBS

Always unset Python environment variables before running GDB inside the ICARUS container, or GDB's embedded Python will fail to load:

```
unset PYTHONPATH PYTHONHOME
```

### C.2 RUNNING GDB

Interactive mode (run and inspect manually)

```
gdb --args lar -c my.fcl -n 1 -s input.root --process-name redo -o /dev/null
(gdb) run
(gdb) bt          # backtrace after crash
(gdb) bt full     # full backtrace with local variables
(gdb) quit
y                # confirm quit
```

Automated mode (log and exit)

```
gdb -ex "set logging file gdb_bt.log" \
    -ex "set logging on" \
    -ex "run" \
    -ex "bt" \
    -ex "bt full" \
    -ex "quit" \
    --args lar -c my.fcl -n 1 -s input.root --process-name redo -o /dev/null
2>&1
```

### C.3 KEY GDB COMMANDS

| Command         | Effect                                              |
|-----------------|-----------------------------------------------------|
| run             | Start execution of the program                      |
| bt              | Print backtrace (call stack at crash point)         |
| bt full         | Print backtrace with local variables for each frame |
| frame N         | Switch to stack frame N                             |
| info locals     | Print all local variables in the current frame      |
| print VAR       | Print the value of a variable                       |
| print *ptr      | Dereference and print the value at a pointer        |
| break FUNC      | Set a breakpoint at the start of a function         |
| break FILE:LINE | Set a breakpoint at a specific file and line number |
| continue        | Continue execution after a breakpoint               |
| next            | Execute the next line (step over function calls)    |

| Command            | Effect                                           |
|--------------------|--------------------------------------------------|
| step               | Execute the next line (step into function calls) |
| info args          | Print function arguments in the current frame    |
| list               | Show source code around the current line         |
| quit               | Exit GDB (confirm with y if a process is active) |
| set logging file F | Redirect all GDB output to file F                |
| set logging on     | Enable logging to the file set above             |

## C.4 READING A BACKTRACE

A backtrace shows the chain of function calls leading to the crash, starting from frame #0 (the innermost call, where the crash occurred) upwards. Example:

```
#0  _int_malloc ()                <- crash here: heap allocator corrupted
#1  malloc ()
#2  operator new ()
#3  art::detail::IPRHelper<...>    <- FindManyP construction
#4  ICARUSFilteredNuSliceHitsProducer::produce
#5  art::EDProducer::doEvent
...
```

In heap corruption crashes, the crash site (#0) is almost always inside malloc or \_int\_malloc. The actual bug (the bad write) happened earlier, in a higher-numbered frame. Look for the first frame that references your own code or art association code.

## C.5 COMMON PATTERNS

Heap corruption (the crash site != the bug site)

```
#0  _int_malloc      <- crash site (heap bookkeeping corrupted)
#1  malloc
#N  YourModule::produce <- the actual bad write happened somewhere upstream
```

Heap corruption: use valgrind to find the actual write site. GDB only shows where the corrupted heap was first accessed, not where it was corrupted.

Null pointer dereference

```
#0  SomeFunction (this=0x0 ...)    <- this=0x0 means null pointer
```

Out-of-bounds vector access

```
#0  std::vector<...>::operator[]    <- usually shows size vs index
    index = 4532, size = 200        <- visible with bt full or print
```

Variables show as <optimised out>

This happens in prof (optimised) builds. Rebuild with -q e26:debug using mrb newDev -T debug\_build -q e26:debug -f to get full variable visibility.

## C.6 SETTING BREAKPOINTS FOR NUGRAPH DEBUGGING

```
# Break at the start of FilterDecoder::writeToEvent:
(gdb) break FilterDecoder::writeToEvent
(gdb) run

# Once stopped, inspect idsmmap size and max key:
(gdb) info locals
(gdb) print idsmmap.size()
(gdb) print size

# Continue to next breakpoint or crash:
(gdb) continue
```